A 签到题

由于 $lcm(x,y)=\frac{xy}{gcd(x,y)}$ ，所以有 $xy \le lcm(x,y)$ ，当且仅当 $gcd(x,y)=1$ 时取等。

所以输出 $1 \sim n$ 即可，答案为 0 。

```
void solve() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i) cout << i << " ";
    cout << "\n";
}
```

B Bob的蛋糕店

设总质量为 $S=\sum_{i=1}^{n} m_i$ ，力矩为 $T=\sum_{i=1}^{n} m_i \cdot i$，质心位置为

$$x=\frac{T}{S}$$

设取走下标的集合为 U ，取走质量为 $s=\sum_{i \in U} m_i$ ，剩余蛋糕的质心不变等价于

$$\frac{T-t}{S-s}=\frac{T}{S}$$

我们用二进制掩码枚举取走的集合即可。

```
void solve() {
    int n, k, S = 0, T = 0;
    cin >> n >> k;
    vector<int> a(n);
    for (int i = 0; i < n; ++i)
        cin >> a[i], S += a[i], T += a[i] * (i + 1);
    for (int mask = 0; mask < (1 << n); ++mask) {
        if (__builtin_popcount(mask) != k)
            continue;
        int s = 0, t = 0;

        for (int i = 0; i < n; ++i) {
            if (mask & (1 << i)) {
                s += a[i];
                t += a[i] * (i + 1);
            }
        }
        if (t * S == T * s) {
            cout << "Yes" << endl;
            return;
        }
    }
    cout << "No" << endl;
}
```

C 修复排列

记录一下每个数字在哪个位置上被修复，然后模拟每个操作修复的路径，如果拆掉的数字不在路径上就继续模拟，否则就对修复无影响。

```
void solve() {
    int n;
    cin >> n;
    vector<int> p(n), id(n), d(n), pos(n), p2(n);
    for (int i = 0; i < n; ++i)
        cin >> p[i], --p[i];
    for (int i = 0; i < n; ++i)
        cin >> id[i], --id[i];
    for (int i = 0; i < n; ++i)
        cin >> d[i], --d[i];
    for (int i = 0; i < n; ++i)
        pos[d[i]] = i;
    for (int i = 0; i < n; ++i)
        p2[i] = pos[p[i]];

    vector<bool> vis(n);
    int cnt = 0;
    for (int i = 0; i < n; ++i) {
        int x = id[i];
        while (!vis[x]) {
            vis[x] = 1;
            ++cnt;
            x = p2[x];
        }
        cout << cnt << " ";
    }
    cout << "\n";
}
```

D 图G

令 $a_i = b_i \cdot x_i$（只考虑满足 $b_i \mid a_i$ 的点，否则该点不可能与任何点连边），有

$$b_j \mid x_i \text{ 且 } b_i \mid x_j$$

按 b 值把点分桶，对每个桶内的每个 x 枚举其所有正因子 d（若 $d \le 1000$），用 $cnt[p][q]$ 统计“有多少个点的 b 等于 p 且对应的 x 可以被 q 整除”。

对每个点 $(u, b_i), (v = x_i)$ 枚举 v 的所有因子 d：如果存在点的 $b = d$ 且该点的 x 被 u 整除（即 $cnt[d][u]$），那么该点与当前点满足条件。把这些计数累加，减去自匹配，再最后除以 2（每条边被算两次）即可。

```
void solve() {
    int n;
    cin >> n;
    vector<int> a(n), b(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < n; ++i) cin >> b[i];
    vector<vector<int>> mp(1001);
```

```
    vector<PII> pr;
    for (int i = 0; i < n; ++i) {
        if (a[i] % b[i] != 0) continue;
        int x = a[i] / b[i];
        mp[b[i]].pb(x);
        pr.eb(b[i], x);
    }
    vector<vector<int>> cnt(1001, vector<int>(1001));
    for (int i = 1; i <= 1000; ++i) {
        if (mp[i].empty()) continue;
        for (ll x : mp[i]) {
            int lim = min((int)floor(sqrt(1.0 * x)), 1000);
            for (int d = 1; d <= lim; ++d) {
                if (x % d == 0) {
                    cnt[i][d]++;
                    int o = x / d;
                    if (o != d && o <= 1000) cnt[i][o]++;
                }
            }
        }
    }
    ll tot = 0;
    for (auto [u, v] : pr) {
        int c = 0, lim = min((int)floor(sqrt(1.0 * v)), 1000);
        for (ll d = 1; d <= lim; ++d) {
            if (v % d == 0) {
                c += cnt[d][u];
                int o = v / d;
                if (o != d && o <= 1000) {
                    c += cnt[o][u];
                }
            }
        }
        c -= (v % u == 0);
        tot += c;
    }

    cout << (tot / 2) << endl;
}
```

E 小R的min

显然有 绝对值和在中位数处最小。

因此问题转化为 查询区间 $[L, R]$ 内所有子区间最小值的中位数

法1：

所以我们记录 $a[i]$ 左边第一个小于 $a[i]$ 的位置，右边第一个小于等于 $a[i]$ 的位置。（因为 n 不大，所以直接暴力即可，不用单调栈）

对一次询问 $[l, r]$，若 $i \in [l, r]$，以 $a[i]$ 作为最小值的子区间个数为：

$$\text{num}_i = (i - \max(l, L[i] + 1) + 1) \times (\min(r, R[i] - 1) - i + 1)$$

子区间总数 $tot = \frac{len \cdot (len+1)}{2}$，中位数排名 $tar = \left\lceil \frac{tot}{2} \right\rceil$

把所有 $a[i]$ 离散化，按值从小到大累加它作为最小值的次数，第一次使前缀和 $\geq tar$ 的值就是答案。

时间复杂度 $O(nm)$

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(n);
    for (int i = 0; i < n; ++i)
        cin >> a[i], b[i] = a[i];
    sort(all(b));
    b.erase(unique(all(b)), b.end());
    vector<int> rk(n);
    for (int i = 0; i < n; ++i) {
        rk[i] = lower_bound(all(b), a[i]) - b.begin();
    }
    vector<int> L(n), R(n);
    for (int i = 0; i < n; ++i) {
        int j = i - 1;
        while (j >= 0 && a[j] >= a[i])
            j--;
        L[i] = j;

        j = i + 1;
        while (j < n && a[j] > a[i])
            j++;
        R[i] = j;
    }
    int k = b.size();
    while (m--) {
        int l, r;
        cin >> l >> r;
        l--, r--;
        int len = r - l + 1;
        ll tar = 1ll * (len * (len + 1) / 2 + 1) / 2;

        vector<int> cnt(k);

        for (int i = l; i <= r; ++i) {
            ll num = 1ll * (i - max(l, L[i] + 1) + 1) * (min(r, R[i] - 1) - i +
1);

            cnt[rk[i]] += num;
        }

        ll cur = 0;
        int ans = b.back();
        for (int i = 0; i < k; ++i) {
            cur += cnt[i];
            if (cur >= tar) {
                ans = b[i];
                break;
            }
        }
        cout << ans << endl;
    }
```

```
}
```

法2：

我们使用二分进行查询，判断答案是否 $\geq v$。如果 $[L, R]$ 内最小值 $\geq v$ 的子区间数量超过了总子区间数的一半，则说明中位数 $\geq v$ 。

直接对每个查询二分答案的复杂度较高。我们使用整体二分，将所有查询一起处理。

因为数组中 $\geq v$ 的元素会形成若干个连续的段。只有完全包含在这些段内部的子区间，其最小值才 $\geq v$ 。
对于每个查询，我们只需计算它与这些段的交集，累加交集内包含的子区间数量即可。

时间复杂度 $O(n^2 logn)$

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(n);
    for (int i = 0; i < n; ++i)
        cin >> a[i], b[i] = a[i];
    sort(all(b));
    b.erase(unique(all(b)), b.end());
    int k = b.size();

    vector<PII> qs(m);
    vector<int> ans(m), L(m), R(m, k - 1);

    auto S = [&](int x) {
        return x <= 0 ? 0ll : 1ll * x * (x + 1) / 2;
    };

    auto calc = [&](int a, int b, int c, int d) {
        return S(min(b, d) - max(a, c) + 1);
    };

    for (int i = 0; i < m; ++i)
        cin >> qs[i].fi >> qs[i].se, qs[i].fi--, qs[i].se--;

    for (int _ = 0; _ < 10; ++_) {
        bool f = false;
        vector<vector<int>> mid(k);

        for (int i = 0; i < m; ++i) {
            if (L[i] <= R[i]) {
                mid[(L[i] + R[i] + 1) / 2].pb(i);
                f = true;
            }
        }
        if (!f)
            break;

        for (int i = 0; i < k; ++i) {
            if (mid[i].empty())
                continue;
            int v = b[i];
```

```
            vector<PII> seg;
            vector<ll> pre;
            pre.pb(0);

            for (int j = 0; j < n;) {
                if (a[j] >= v) {
                    int st = j;
                    while (j < n && a[j] >= v)
                        j++;
                    ll w = S(j - st);
                    seg.eb(st, j - 1);
                    pre.pb(pre.back() + w);
                } else {
                    j++;
                }
            }

            for (int id : mid[i]) {
                auto [l, r] = qs[id];
                ll tar = (S(r - l + 1) + 1) / 2;
                ll cnt = 0;

                int il = lower_bound(all(seg), l, [&](auto a, int v) { return
a.se < v; }) - seg.begin();
                int ir = upper_bound(all(seg), r, [&](int v, auto b) { return v <
b.fi; }) - seg.begin() - 1;

                if (il <= ir) {
                    if (il == ir) {
                        cnt += calc(seg[il].fi, seg[il].se, l, r);
                    } else {
                        cnt += calc(seg[il].fi, seg[il].se, l, r);
                        cnt += calc(seg[ir].fi, seg[ir].se, l, r);
                        if (il + 1 <= ir - 1)
                            cnt += pre[ir] - pre[il + 1];
                    }
                }

                if (cnt >= tar) {
                    ans[id] = v;
                    L[id] = i + 1;
                } else {
                    R[id] = i - 1;
                }
            }
        }
    }

    for (auto x : ans)
        cout << x << endl;
}
```

F 种树

瞪眼法看出 $\sum\limits_{k=1}^{\infty}\left\lfloor\frac{i\cdot j+2^k}{2^{k+1}}\right\rfloor=i\cdot j$，于是题目转化为求所有满足 $1\le i,j\le n$ 且 $gcd(i,j)=1$ 的种子位置的 $i\cdot j$ 之和。

这个是经典莫反题，接下来给出推导：

$$
\begin{aligned}
S&=\sum_{i=1}^{n}\sum_{j=1}^{n}[gcd(i,j)=1](i\cdot j)\\
&=\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{d|i,d|j}\mu(d)ij\\
&\overset{i=d\cdot a,j=d\cdot b}{=}\sum_{d=1}^{n}\mu(d)d^2\left(\sum_{a=1}^{\lfloor\frac{n}{d}\rfloor}a\right)\left(\sum_{b=1}^{\lfloor\frac{n}{d}\rfloor}b\right)\\
&\overset{S(n)=\frac{(1+n)n}{2}}{=}\sum_{d=1}^{n}\mu(d)d^2(S(\left\lfloor\frac{n}{d}\right\rfloor))^2
\end{aligned}
$$

因为 $n\le 10^9$，所以使用杜教筛求前缀和（预处理 2×10^6 个前缀和，剩余部分记忆化），用数论分块进行求和即可。

```
constexpr int maxn = 2e6;

Z Sn(Z n) {
    return n * (n + 1) / 2;
}

void solve() {
    int n;
    cin >> n;

    Z ans = 0;

    for (int l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans += (sumMu(r) - sumMu(l - 1)) * Sn(Z(n / l)) * Sn(Z(n / l));
    }
    cout << ans << endl;
}
```

头文件

```
#include <bits/stdc++.h>
#define pb push_back
#define pf push_front
#define pob pop_back
#define pof pop_front
#define eb emplace_back
#define fi first
#define se second
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
```

```
#define endl "\n"
using namespace std;

using ll = long long;
using ld = long double;
using ui = unsigned;
using ull = unsigned long long;
using i128 = __int128;
using PII = pair<int, int>;
using PLL = pair<ll, ll>;
using TIII = tuple<int, int, int>;
using TLLL = tuple<ll, ll, ll>;

void init() {
}

void solve() {
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    init();

    int t = 1;
    cin >> t;
    cout << fixed << setprecision(15);
    for (int _ = 1; _ <= t; ++_) {
        solve();
    }

    return 0;
}
```

自动取模

```
template <class T>
constexpr T ksm(T a, ll b) {
    T res = 1;
    while (b) {
        if (b & 1)
            res *= a;
        b >>= 1;
        a *= a;
    }
    return res;
}

template <int P>
struct MInt {
    int x;
    constexpr MInt() : x() {}
    constexpr MInt(ll x) : x{norm(x % getMod())} {}
```

```
    static int Mod;
    constexpr static int getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(int Mod_) {
        Mod = Mod_;
    }
    constexpr int norm(int x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr int val() const {
        return x;
    }
    explicit constexpr operator int() const {
        return x;
    }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
        return res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return ksm(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
        x = 1LL * x * rhs.x % getMod();
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & {
        return *this *= rhs.inv();
    }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
```

```
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr istream &operator>>(istream &is, MInt &a) {
        ll v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr ostream &operator<<(ostream &os, const MInt &a) {
        return os << a.val();
    }
    friend constexpr bool operator==(MInt lhs, MInt rhs) {
        return lhs.val() == rhs.val();
    }
    friend constexpr bool operator!=(MInt lhs, MInt rhs) {
        return lhs.val() != rhs.val();
    }
};

template <>
int MInt<0>::Mod = 998244353;

template <int V, int P>
constexpr MInt<P> CInv = MInt<P>(V).inv();

using Z = MInt<MOD>;
```

莫比乌斯函数&杜教筛（对 d^2 求和）

```
Z Sn2(Z n) {
    return n * (n + 1) * (2 * n + 1) / 6;
}

unordered_map<int, Z> fMu;

vector<int> spf(maxn + 1), primes, mu(maxn + 1);
vector<Z> smu(maxn + 1);

Z sumMu(int n) {
    if (n <= maxn) {
        return smu[n];
    }
```

```
    if (fMu.count(n)) {
        return fMu[n];
    }
    if (n == 0) {
        return 0;
    }
    Z ans = 1;
    int l = 2, r;
    while (l <= n) {
        r = n / (n / l);
        ans -= (Sn2(Z(r)) - Sn2(Z(l - 1))) * sumMu(n / l);
        l = r + 1;
    }
    return fMu[n] = ans;
}

void init() {
    mu[1] = 1, smu[1] = 1;

    for (int i = 2; i <= maxn; i++) {
        if (!spf[i])
            spf[i] = i, mu[i] = -1, primes.push_back(i);

        for (int j = 0; primes[j] * i <= maxn; j++) {
            int m = primes[j] * i;
            spf[m] = primes[j];

            if (i % primes[j] == 0) {
                mu[m] = 0;
                break;
            }
            mu[m] = -mu[i];
        }

        if (mu[i] == 1)
            smu[i] = smu[i - 1] + Z(i) * i;
        else if (mu[i] == -1)
            smu[i] = smu[i - 1] - Z(i) * i;
        else
            smu[i] = smu[i - 1];
    }
}
```